



Complete Technology Deep-Dive

Learn Every Feature + The Tech Behind It

React 19	Node.js	MongoDB	JWT Auth	Gemini AI	TailwindCSS
----------	---------	---------	----------	-----------	-------------

A feature-by-feature breakdown of every technology used to build a full-stack AI-powered portfolio generator — with tutorials for each.

buildfolio-one.vercel.app · github.com/Mdhummad/buildfolio

TABLE OF CONTENTS

01	Project Overview — What is Buildfolio?
02	Feature 1: 6-Step Portfolio Builder → React 19 + React Hook Form
03	Feature 2: Client-Side Routing → React Router DOM v7
04	Feature 3: UI Styling → TailwindCSS v3 + Custom CSS (Glassmorphism)
05	Feature 4: REST API Backend → Node.js + Express
06	Feature 5: Database Storage → MongoDB Atlas + Mongoose
07	Feature 6: Authentication System → JWT + bcryptjs
08	Feature 7: HTTP Communication → Axios + Interceptors
09	Feature 8: AI Resume Optimizer → Google Gemini API
10	Feature 9: ZIP Export → Archiver (Node.js)
11	Feature 10: Password Reset Emails → Nodemailer
12	Feature 11: State Management → React Context API
13	Feature 12: Deployment → Vercel + Render + MongoDB Atlas
14	Feature 13: Security Best Practices → CORS, Env Vars, Hashing
15	Learning Roadmap — What to Study Next

Buildfolio is a **full-stack web application** that lets any developer, designer, or student create a professional portfolio website and ATS-optimized resume in minutes — no design or coding skills required.

Frontend	React 19 · React Router v7 · TailwindCSS · Axios · Vite 8
Backend	Node.js · Express · Mongoose · bcryptjs · JWT · Nodemailer · Archiver
Database	MongoDB Atlas (cloud-hosted NoSQL)
AI Layer	Google Gemini 1.5 Flash (resume optimization)
Hosting	Vercel (frontend) · Render (backend) · MongoDB Atlas (DB)
Auth	JWT tokens (7-day expiry) + bcryptjs password hashing
Export	Portfolio downloaded as a deployable ZIP (HTML + CSS + JS)

How Data Flows Through the App

Understanding the full data flow is key before diving into individual technologies:

- User fills the 6-step Builder wizard (React frontend)
- Data is held in PortfolioContext (React Context API) in memory
- On 'Save Progress', Axios sends a PUT request to Express backend
- Express validates the JWT token, then Mongoose saves data to MongoDB Atlas
- On the Preview page, the chosen template is populated with real user data
- ATS Optimizer: user pastes a job description → backend calls Gemini AI → returns score + tips
- ZIP Export: Express uses Archiver to package HTML/CSS/JS → streams it as a download

■ The Feature

The Builder Page is a multi-step wizard where users fill in their portfolio data across 6 organized steps: Personal → Experience → Education → Skills → Projects → Links & Achievements. Each step has form validation, and data is preserved as users move between steps.

■ Technology: React 19

React is a **JavaScript library for building user interfaces**. It was created by Meta (Facebook) and is the most popular frontend framework in the world. Buildfolio uses React 19, the latest major version.

Core Concepts You Must Learn

Components	Everything in React is a component — a reusable piece of UI. BuilderPage.jsx, Navbar.jsx, AuthModal.jsx
JSX	JSX lets you write HTML-like syntax inside JavaScript. React compiles it to regular JS. You write <div cl
Props	Props (properties) are how you pass data from a parent component to a child. Like function arguments k
State (useState)	State is data that, when changed, causes the component to re-render (update the UI). The useState hoc
useEffect	Runs code after the component renders. Used for: fetching data from an API, setting up event listeners,
Conditional Rendering	React can show or hide elements based on conditions. In Buildfolio, only the current wizard step is show
Lists & Keys	When rendering arrays (like a list of work experiences), React needs a unique key prop on each elemen

How Buildfolio Uses React

- Each of the 6 wizard steps is a separate component rendered conditionally based on currentStep state
- The Builder page tracks currentStep with useState — clicking Next increments it
- Profile photo upload uses useState to store the base64 image string
- Dynamic arrays (work experiences, projects) use map() to render each entry
- useEffect is used to load saved portfolio data from the server after login

Code Example: Step Navigation in React

```
// BuilderPage.jsx – simplified
import { useState } from "react";

function BuilderPage() {
  const [currentStep, setCurrentStep] = useState(1);

  return (
    <div>
      {/* Progress Bar */}
```

```

<div className="progress-bar">Step {currentStep} of 6</div>

{/* Render only the active step */}
{currentStep === 1 && <PersonalStep />}
{currentStep === 2 && <ExperienceStep />}
{currentStep === 3 && <EducationStep />}
{currentStep === 4 && <SkillsStep />}
{currentStep === 5 && <ProjectsStep />}
{currentStep === 6 && <LinksStep />}

{/* Navigation Buttons */}
<button onClick={() => setCurrentStep(s => s - 1)}>Back</button>
<button onClick={() => setCurrentStep(s => s + 1)}>Next</button>
</div>
);
}

```

■ Technology: React Hook Form

React Hook Form (RHF) is a library that makes managing forms in React easy and performant. Without it, you would need to manually track every input's value with `useState`, which causes the entire component to re-render on every keystroke.

Why RHF is Better Than Manual State

Problem with manual state	Every keystroke calls <code>setState</code> → entire form re-renders → slow
RHF solution	Uses uncontrolled inputs + refs → only re-renders on submit/validation
register()	Connects an input to the form. Replaces <code>value={}</code> + <code>onChange={}</code> manually
handleSubmit()	Wraps your submit handler. Auto-validates before calling your function
formState.errors	Object containing all validation errors. Used to show error messages
watch()	Observes a field's value in real-time. Used for conditional fields

```

// ExperienceStep.jsx – React Hook Form usage
import { useForm, useFieldArray } from "react-hook-form";

function ExperienceStep() {
  const { register, handleSubmit, control, formState: { errors } } = useForm();

  // useFieldArray manages dynamic lists (add/remove entries)
  const { fields, append, remove } = useFieldArray({
    control,
    name: "experiences"
  });

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      {fields.map((field, index) => (
        <div key={field.id}>
          <input

```

```

{...register(`experiences.${index}.company`, {
  required: "Company name is required"
})}
placeholder="Company Name"
/>
{errors.experiences?.[index]?.company && (
  <span>{errors.experiences[index].company.message}</span>
)}
<button type="button" onClick={() => remove(index)}>Remove</button>
</div>
)}}
<button type="button" onClick={() => append({ company: "", role: "" })}>
+ Add Experience
</button>
</form>
);
}

```

■ **Pro Tip:** useFieldArray is perfect for Buildfolio's dynamic Experience, Education, and Projects sections where users can add unlimited entries.

■ The Feature

Buildfolio has 4 distinct pages: Landing (/) → Builder (/builder) → Templates (/templates) → Preview (/preview). Navigating between them should feel instant, without full page reloads. That is what React Router enables.

■ Technology: React Router DOM v7

React Router is the standard routing library for React. In a traditional website, clicking a link sends a request to the server and loads a new HTML page. In a Single Page Application (SPA) like Buildfolio, React Router intercepts navigation and renders the correct component without any server round-trip — making the app feel lightning fast.

Key Concepts

SPA (Single Page App)	One HTML file loads once. JS handles all navigation. No page reloads.
BrowserRouter	Wraps the entire app. Gives all children access to routing features.
Routes + Route	Declares which component to render for which URL path.
<Link>	Like <a href> but prevents page reload. Updates the URL and renders the component.
useNavigate()	Hook to programmatically navigate. Used after form submit or login.
useParams()	Extracts URL parameters like /portfolio/:id → id = 'abc123'.
Protected Routes	A pattern to redirect unauthenticated users away from private pages.

```
// App.jsx – Buildfolio routing setup
import { BrowserRouter, Routes, Route } from "react-router-dom";
import LandingPage from "../pages/LandingPage";
import BuilderPage from "../pages/BuilderPage";
import TemplatePage from "../pages/TemplatePage";
import PreviewPage from "../pages/PreviewPage";

function App() {
  return (
    <BrowserRouter>
      <Navbar /> {/* shown on all pages */}
      <Routes>
        <Route path="/" element={<LandingPage />} />
        <Route path="/builder" element={<BuilderPage />} />
        <Route path="/templates" element={<TemplatePage />} />
        <Route path="/preview" element={<PreviewPage />} />
      </Routes>
    </BrowserRouter>
  );
}

// Navigating programmatically after completing wizard
```



```
import { useNavigate } from "react-router-dom";

function FinalStep() {
  const navigate = useNavigate();

  const handleFinish = () => {
    navigate("/templates"); // ← goes to template selection
  };

  return <button onClick={handleFinish}>Choose Template</button>;
}
```

■ **Note:** Buildfolio also has a `vercel.json` with a rewrite rule: all requests to `/*` are served the same `index.html`. This is required for SPAs deployed on Vercel — without it, refreshing `/builder` would return a 404.

■ The Feature

Buildfolio has a distinctive dark design system with glassmorphism cards, glowing cyan accents, gradient text, and smooth animations. The UI is polished enough to look like a commercial product.

■ Technology: TailwindCSS v3

Tailwind CSS is a **utility-first CSS framework**. Instead of writing custom CSS class names like `.card` or `.hero-title`, you apply small, single-purpose utility classes directly in your HTML/JSX.

Traditional CSS	You write: <code>.card { padding: 16px; background: white; border-radius: 8px; }</code>
Tailwind approach	<code><div className="p-4 bg-white rounded-lg"></code> — same result, no CSS file needed
Responsive	Add breakpoint prefix: <code>md:text-lg</code> (medium screens), <code>lg:grid-cols-3</code> (large screens)
Hover/focus states	<code>hover:bg-cyan-500</code> , <code>focus:ring-2</code> , <code>active:scale-95</code> — interactivity without JS
Dark mode	<code>dark:bg-slate-900 dark:text-white</code> — switch styles based on theme
Customization	<code>tailwind.config.js</code> lets you define custom colors, fonts, spacing

```
// Example: Buildfolio button using Tailwind
<button className="
px-6 py-3
bg-cyan-500 hover:bg-cyan-400
text-white font-bold text-sm
rounded-full
transition-all duration-200
shadow-lg hover:shadow-cyan-500/40
active:scale-95
">
Download ZIP
</button>

// Grid layout – 1 column on mobile, 3 on desktop
<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
{templates.map(t => <TemplateCard key={t.id} {...t} />)}
</div>
```

■ Technology: Custom CSS — Glassmorphism Design System

Tailwind has limitations for complex effects. Buildfolio's custom `index.css` defines the advanced design system: glassmorphism, glowing shadows, gradient text, and keyframe animations.

Glassmorphism — Explained

Glassmorphism is a design trend that makes UI cards look like frosted glass — semi-transparent background, blur effect, and a subtle border. It requires three CSS properties working together:

```
/* index.css – Glassmorphism card */
.glass-card {
background: rgba(255, 255, 255, 0.05); /* 5% white = semi-transparent */
backdrop-filter: blur(12px); /* blurs what's BEHIND the card */
-webkit-backdrop-filter: blur(12px); /* Safari support */
border: 1px solid rgba(255, 255, 255, 0.1); /* subtle white border */
border-radius: 16px;
}

/* Cyan glow effect (multi-layer box-shadow) */
.glow-cyan {
box-shadow:
0 0 15px rgba(34, 211, 238, 0.3), /* close soft glow */
0 0 40px rgba(34, 211, 238, 0.15), /* wider ambient glow */
0 0 80px rgba(34, 211, 238, 0.05); /* faint outer glow */
}

/* Gradient text – cyan to gold */
.gradient-text {
background: linear-gradient(135deg, #22d3ee, #fbbf24);
-webkit-background-clip: text; /* clip background to text shape */
-webkit-text-fill-color: transparent; /* make text transparent = shows gradient */
}

/* Floating animation for hero cards */
@keyframes float {
0%, 100% { transform: translateY(0px); }
50% { transform: translateY(-12px); } /* moves up 12px */
}

.float-animation {
animation: float 4s ease-in-out infinite; /* loops forever */
}
```

■ **Pro Tip:** backdrop-filter: blur() only works when the element has a transparent or semi-transparent background. A fully opaque card won't show any blur.

The backend is the server that handles everything the frontend can't do alone: storing data in MongoDB, verifying passwords, generating JWTs, calling Gemini AI, and building ZIP files. It exposes a REST API that the frontend communicates with via Axios.

Node.js is a **JavaScript runtime environment** that lets you run JavaScript code outside the browser — on a server. Before Node.js, JavaScript only ran in browsers. Node.js changed that by using Chrome's V8 engine.

Event-driven	Node.js handles requests using an event loop. It doesn't block while waiting for I/O — very efficient for APIs.
npm	Node Package Manager — like pip for Python. Install packages with: <code>npm install express</code>
CommonJS/ESM	Node.js supports both <code>require()</code> (CommonJS) and <code>import/export</code> (ES Modules). Buildfolio uses <code>require()</code> .
package.json	Like a manifest. Lists your project name, version, scripts (<code>npm start</code> , <code>npm run dev</code>), and all dependencies.
dotenv	Loads environment variables from a <code>.env</code> file into <code>process.env</code> . Keeps secrets out of code.

Express is a **minimal web framework for Node.js**. It gives you routing, middleware, request/response handling, and more — with very little boilerplate.

[illegible]

```
app.get("/health", (req, res) => {  
  res.json({ status: "ok", time: new Date(), db: "connected" });  
});  
  
// ■■ Start Server ████████████████████████████████████████████████████████  
const PORT = process.env.PORT || 5000;  
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

Understanding Middleware

Middleware is a function that runs between the request arriving and the response being sent. Every request passes through a chain of middleware functions. In Buildfolio:

- `express.json()` — parses the JSON body of POST/PUT requests
- `cors()` — adds CORS headers so the browser allows cross-origin requests
- `auth.js` middleware — verifies the JWT token on protected routes

```
// middleware/auth.js - JWT verification middleware
const jwt = require("jsonwebtoken");

module.exports = (req, res, next) => {
  // Get token from Authorization header: "Bearer eyJhbG..."
  const token = req.headers.authorization?.split(" ")[1];

  if (!token) {
    return res.status(401).json({ message: "No token provided" });
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded; // Attach user info to request
    next(); // Call next middleware / route handler
  } catch (err) {
    return res.status(401).json({ message: "Invalid token" });
  }
};

// routes/portfolio.js - using the auth middleware
const auth = require("../middleware/auth");

router.get("/", auth, async (req, res) => {
  // req.user.id is available because auth middleware set it
  const portfolio = await Portfolio.findOne({ user: req.user.id });
  res.json(portfolio);
});
```

■ The Feature

Every user's portfolio data — name, experience, projects, skills, social links, profile photo (base64) — is stored in the cloud database. Users can log in from any device and find their data exactly where they left it.

■ Technology: MongoDB Atlas

MongoDB is a **NoSQL document database**. Unlike SQL databases (MySQL, PostgreSQL) that store data in rigid tables with fixed columns, MongoDB stores data as flexible JSON-like documents. This makes it perfect for Buildfolio's portfolio data, which has nested arrays (experiences, projects) and optional fields.

SQL databases	Data in rigid tables. Every row has the same columns. Schema must be designed upfront.
MongoDB	Data in flexible documents (JSON-like). Each document can have different fields.
Collection	MongoDB's equivalent of a SQL table. Buildfolio has: users collection + portfolios collection.
Document	One record. A user document = { name, email, password, resetToken }.
MongoDB Atlas	Cloud-hosted MongoDB. Managed service — no server setup needed. Free tier available.
BSON / ObjectId	MongoDB stores data as BSON (Binary JSON). Each document gets an auto-generated <code>_id</code> (ObjectId).

■ Technology: Mongoose

Mongoose is an **ODM (Object Document Mapper)** for MongoDB in Node.js. It adds structure, validation, and useful methods on top of MongoDB's raw driver. Without Mongoose you'd write raw MongoDB queries — Mongoose makes it much more readable.

```
// models/User.js — Mongoose Schema + Model
const mongoose = require("mongoose");
const bcrypt = require("bcryptjs");

const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  password: { type: String, required: true, minlength: 6 },
  resetToken: { type: String, default: null },
  resetExpiry: { type: Date, default: null },
}, { timestamps: true }); // auto-adds createdAt, updatedAt

// ■■ Pre-save hook: hash password before saving ■■■■■■■■■■
userSchema.pre("save", async function(next) {
  if (!this.isModified("password")) return next(); // skip if not changed
  this.password = await bcrypt.hash(this.password, 12);
  next();
});

// ■■ Instance method: compare passwords ■■■■■■■■■■■■■■■■■■■■
```

```

userSchema.methods.comparePassword = async function(candidate) {
  return bcrypt.compare(candidate, this.password);
};

module.exports = mongoose.model("User", userSchema);

// models/Portfolio.js – Nested arrays and objects
const portfolioSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  personalInfo: {
    name: String, title: String, summary: String,
    email: String, phone: String, location: String,
    website: String, avatar: String // stored as base64 string
  },
  experiences: [{
    company: String, role: String,
    startDate: String, endDate: String,
    location: String, description: String
  }],
  skills: {
    technical: [String],
    languages: [String],
    tools: [String],
    soft: [String]
  },
  projects: [{
    name: String, description: String,
    techStack: String, demoUrl: String,
    githubUrl: String, featured: Boolean
  }],
});

```

CRUD Operations with Mongoose

```

// routes/portfolio.js – GET and PUT handlers
const Portfolio = require("../models/Portfolio");

// GET: Fetch user's portfolio
router.get("/", auth, async (req, res) => {
  try {
    const portfolio = await Portfolio.findOne({ user: req.user.id });
    if (!portfolio) return res.status(404).json({ message: "Not found" });
    res.json(portfolio);
  } catch (err) {
    res.status(500).json({ message: "Server error" });
  }
});

// PUT: Save/update portfolio (upsert = create if not exists)
router.put("/", auth, async (req, res) => {
  try {
    const portfolio = await Portfolio.findOneAndUpdate(
      { user: req.user.id }, // find by user id
      { ...req.body }, // update with request body
    );
  }
});

```

```
{ new: true, upsert: true } // upsert: true = create if not found
);
res.json(portfolio);
} catch (err) {
res.status(500).json({ message: "Server error" });
}
});
```


■ The Feature

Users can register an account, log in, and stay logged in for 7 days without re-entering their password. The backend verifies who you are on every protected request. Passwords are never stored in plain text.

■ Technology: JWT (JSON Web Tokens)

JWT is an open standard for securely transmitting information between parties as a JSON object. In Buildfolio, JWTs replace traditional sessions — the server doesn't need to remember who is logged in. The token itself carries the proof.

How a JWT Works

A JWT looks like: `eyJhbGciOiJIUzI1NiJ9.eyJpZCI6IjEyMyJ9.XYZ`

It has 3 parts separated by dots:

- **Header** — algorithm used (HS256) and token type (JWT), base64 encoded
- **Payload** — the data you stored (user ID, expiry), base64 encoded
- **Signature** — HMAC of header + payload using your JWT_SECRET. Tamper-proof.

If anyone modifies the payload (e.g. changes their user ID), the signature won't match — the server rejects it. The secret key is only on the server, so nobody can forge a valid token.

```
// routes/auth.js — Register: create user, return JWT
const jwt = require("jsonwebtoken");
const User = require("../models/User");

router.post("/register", async (req, res) => {
  const { name, email, password } = req.body;
  try {
    const existing = await User.findOne({ email });
    if (existing) return res.status(400).json({ message: "Email taken" });

    const user = await User.create({ name, email, password });
    // Password is auto-hashed by the pre-save hook in User.js

    const token = jwt.sign(
      { id: user._id }, // payload — only store user ID
      process.env.JWT_SECRET, // secret key from .env
      { expiresIn: "7d" } // token valid for 7 days
    );

    res.status(201).json({ token, user: { id: user._id, name, email } });
  } catch (err) {
    res.status(500).json({ message: "Server error" });
  }
});
```

```

});

// Login: verify password, return JWT
router.post("/login", async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (!user) return res.status(401).json({ message: "Invalid credentials" });

  const valid = await user.comparePassword(password);
  if (!valid) return res.status(401).json({ message: "Invalid credentials" });

  const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: "7d" });
  res.json({ token, user: { id: user._id, name: user.name, email } });
});

```

Frontend: Storing and Using the Token

```

// AuthContext.jsx – login function
const login = (token, user) => {
  localStorage.setItem("pb_token", token); // persist token in browser
  localStorage.setItem("pb_user", JSON.stringify(user));
  setIsAuthenticated(true);
  setUser(user);
};

const logout = () => {
  localStorage.removeItem("pb_token");
  localStorage.removeItem("pb_user");
  setIsAuthenticated(false);
};

// utils/api.js – Axios interceptor adds token to every request
api.interceptors.request.use(config => {
  const token = localStorage.getItem("pb_token");
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Auto-logout on 401
api.interceptors.response.use(null, error => {
  if (error.response?.status === 401) {
    localStorage.clear();
    window.location.href = "/";
  }
  return Promise.reject(error);
});

```

■ Technology: bcryptjs

bcryptjs hashes passwords before storing them. Hashing is a one-way function — you can't reverse a hash to get the original password. When a user logs in, bcrypt hashes their input and compares it to the stored hash.

```
const bcrypt = require("bcryptjs");

// Hash a password (cost factor 12 = 2^12 rounds of hashing)
const hashed = await bcrypt.hash("mypassword123", 12);
// hashed = "$2a$12$..." – stored in MongoDB

// Verify at login – returns true or false
const isMatch = await bcrypt.compare("mypassword123", hashed); // true
const isMatch2 = await bcrypt.compare("wrongpassword", hashed); // false
```

■ **Note:** The cost factor (12) controls how slow the hash is. Higher = slower = harder to brute-force. bcryptjs automatically includes a random salt in the hash, so two users with the same password get different hashes.

■ The Feature

Every time Buildfolio's frontend talks to the backend — fetching portfolio data, saving progress, calling the AI optimizer, downloading a ZIP — it uses HTTP requests. Axios is the library that makes these requests and handles responses.

■ Technology: Axios

Axios is a promise-based HTTP client for JavaScript. It works in both browsers and Node.js. While the browser has a native `fetch()` API, Axios is preferred in production apps because it has automatic JSON parsing, request/response interceptors, cleaner error handling, and timeout support.

<code>axios.get(url)</code>	Fetch data. Used to load the saved portfolio from MongoDB.
<code>axios.post(url, data)</code>	Send data. Used for login, register, ATS optimization.
<code>axios.put(url, data)</code>	Update data. Used to save/update portfolio data.
<code>Interceptors</code>	Functions that run before every request or after every response. Like middleware for HTTP.
<code>Base URL</code>	Set once: <code>axios.create({ baseURL: 'https://api.com' })</code> . All calls use this prefix.
<code>Async/Await</code>	Axios returns Promises. Use <code>async/await</code> for cleaner code than <code>.then()</code> chains.

```
// utils/api.js – Buildfolio's Axios instance
import axios from "axios";

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || "https://buildfolio-t1vl.onrender.com",
  timeout: 30000, // 30 second timeout
});

// ■■ Request Interceptor: attach JWT to every request ■■■
api.interceptors.request.use(config => {
  const token = localStorage.getItem("pb_token");
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// ■■ Response Interceptor: handle 401 globally ■■■■■■■■■■
api.interceptors.response.use(
  response => response,
  error => {
    if (error.response?.status === 401) {
      localStorage.removeItem("pb_token");
      localStorage.removeItem("pb_user");
      window.location.href = "/"; // redirect to home
    }
    return Promise.reject(error);
  }
);
```

[illegible]

■ The Feature

The ATS Optimizer is Buildfolio's most impressive feature. A user pastes any job description, clicks Analyze, and within seconds receives: an ATS score (0–100), matched keywords, missing keywords, an optimized professional summary, reordered skills, and experience enhancement tips.

■ Technology: Google Gemini 1.5 Flash API

Google Gemini is a family of large language models (LLMs) made by Google DeepMind. Buildfolio uses Gemini 1.5 Flash — a fast, efficient model well-suited for structured tasks like resume analysis. The API is called directly from the Express backend using Node.js's built-in fetch.

What is an LLM / Generative AI?

Large Language Models are AI models trained on massive text datasets. They learn patterns in language and can generate coherent, contextually relevant text. When you give Gemini a prompt, it predicts the most likely continuation — producing responses that seem intelligent.

Prompt Engineering	The art of crafting instructions that guide the model to produce the exact output format you need.
System Prompt	A hidden instruction that sets the AI's role and behavior. Like telling the AI: 'You are an ATS expert.'
Temperature	Controls randomness. 0 = deterministic/focused. 1 = creative/varied. ATS analysis uses low temperature.
Max Tokens	Limits response length. 1000 tokens ≈ 750 words.
JSON mode	Prompting the model to respond ONLY in JSON — making it easy to parse programmatically.
API Key	Required to authenticate with Google's Gemini API. Get one free at aistudio.google.com .

```
// routes/gemini.js – ATS Resume Optimization
router.post("/optimize-resume", auth, async (req, res) => {
  const { portfolioData, jobDescription } = req.body;

  const prompt = `
  You are an expert ATS (Applicant Tracking System) analyst.
  Analyze the following resume against the job description.

  RESUME DATA:
  ${JSON.stringify(portfolioData, null, 2)}

  JOB DESCRIPTION:
  ${jobDescription}

  Respond ONLY with a valid JSON object (no markdown, no backticks):
  {
    "atsScore": <number 0-100>,
    "matchedKeywords": ["keyword1", "keyword2"],
    "missingKeywords": ["keyword3", "keyword4"],
```

```

"optimizedSummary": "<rewritten professional summary>",
"reorderedSkills": {
  "technical": ["most relevant first"],
  "languages": ["most relevant first"]
},
"experienceEnhancements": [
  { "role": "Job Title", "suggestions": ["bullet1", "bullet2"] }
],
"scoreBreakdown": {
  "keywordMatch": 70,
  "experienceMatch": 80,
  "skillsMatch": 65,
  "formatting": 90
}
}`;

const response = await fetch(
  `https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent?key=${process.env.GEMINI_API_KEY}`,
  {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      contents: [{ parts: [{ text: prompt }] }],
      generationConfig: { temperature: 0.3, maxOutputTokens: 2048 }
    })
  }
);

const data = await response.json();
const text = data.candidates[0].content.parts[0].text;
const parsed = JSON.parse(text.replace(/```json|```/g, "").trim());
res.json(parsed);
});

```

■ **Pro Tip:** Always strip Markdown code fences (```) from Gemini's response before `JSON.parse()`. Even when you ask for pure JSON, models sometimes wrap it in fences.

■ The Feature

After choosing a template, users can click 'Download ZIP' to get a ready-to-deploy package: an HTML file (their portfolio), a CSS file, and a JS file. They can upload this ZIP to GitHub Pages, Netlify, or any static host.

■ Technology: Archiver

Archiver is a Node.js library for creating archive files (ZIP, TAR). In Buildfolio, the backend dynamically generates the portfolio HTML/CSS/JS by injecting the user's data into the template, then uses Archiver to package these files into a ZIP stream sent directly to the browser.

```
// routes/export.js – ZIP generation and streaming
const archiver = require("archiver");

router.post("/", auth, async (req, res) => {
  const { portfolioData, templateId } = req.body;

  // Generate HTML content for the chosen template
  const html = generateHTML(portfolioData, templateId);
  const css = generateCSS(templateId);
  const js = generateJS(portfolioData);

  // Set response headers to trigger browser download
  res.setHeader("Content-Type", "application/zip");
  res.setHeader("Content-Disposition", "attachment; filename=portfolio.zip");

  // Create ZIP archive streamed directly to the response
  const archive = archiver("zip", { zlib: { level: 9 } }); // max compression
  archive.pipe(res); // pipe ZIP stream → HTTP response → browser

  // Add files to the archive from strings (no disk I/O needed)
  archive.append(html, { name: "index.html" });
  archive.append(css, { name: "styles.css" });
  archive.append(js, { name: "script.js" });

  // Finalize: flushes remaining data and ends the stream
  await archive.finalize();
});

// Helper: inject user data into HTML template
function generateHTML(data, templateId) {
  return `<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>${data.personalInfo.name} – Portfolio</title>
<link rel="stylesheet" href="styles.css">
</head>
<body>
```



```
<h1>\${data.personalInfo.name}</h1>
<p>\${data.personalInfo.title}</p>
<!-- ... rest of template ... -->
<script src="script.js"></script>
</body>
</html>\`;
}
```

■ **Note:** Archiver streams the ZIP directly to the HTTP response without creating a temporary file on disk. This is memory-efficient and faster — especially on cloud servers like Render with limited storage.

■ The Feature

If a user forgets their password, they enter their email and receive a reset link with a 1-hour expiry. Clicking the link opens a form to set a new password. The link is single-use and time-limited for security.

■ Technology: Nodemailer

Nodemailer is the most popular Node.js library for sending emails. It supports SMTP (Simple Mail Transfer Protocol), which is the standard protocol for email transmission. Buildfolio configures it with Gmail SMTP.

```
// routes/auth.js – Forgot Password flow
const nodemailer = require("nodemailer");
const crypto = require("crypto"); // built-in Node.js module

// Create SMTP transporter (think of it as your email client config)
const transporter = nodemailer.createTransport({
  host: process.env.SMTP_HOST, // smtp.gmail.com
  port: process.env.SMTP_PORT, // 587
  secure: false, // TLS
  auth: {
    user: process.env.SMTP_USER, // your Gmail address
    pass: process.env.SMTP_PASS, // Gmail App Password (not your real password)
  }
});

router.post("/forgot-password", async (req, res) => {
  const { email } = req.body;
  const user = await User.findOne({ email });
  if (!user) return res.status(404).json({ message: "No account found" });

  // Generate a cryptographically secure random token
  const resetToken = crypto.randomBytes(32).toString("hex");
  const resetExpiry = Date.now() + 60 * 60 * 1000; // 1 hour from now

  // Save token + expiry to the user document
  user.resetToken = resetToken;
  user.resetExpiry = resetExpiry;
  await user.save();

  // Send the email
  const resetUrl = `${process.env.FRONTEND_URL}/reset-password?token=${resetToken}`;
  await transporter.sendMail({
    from: `"Buildfolio" <${process.env.SMTP_USER}>`,
    to: email,
    subject: "Reset your Buildfolio password",
    html: `<p>Click to reset: <a href="${resetUrl}">${resetUrl}</a></p>
    <p>This link expires in 1 hour.</p>`
  });
});
```

```
res.json({ message: "Reset email sent" });
});

router.post("/reset-password", async (req, res) => {
  const { token, newPassword } = req.body;

  // Find user with valid, non-expired token
  const user = await User.findOne({
    resetToken: token,
    resetExpiry: { $gt: Date.now() } // token must not be expired
  });

  if (!user) return res.status(400).json({ message: "Invalid or expired token" });

  user.password = newPassword; // pre-save hook hashes it
  user.resetToken = null; // invalidate token (single-use)
  user.resetExpiry = null;
  await user.save();

  res.json({ message: "Password reset successful" });
});
```

■ **Pro Tip:** Never use your real Gmail password for SMTP. Generate a Gmail App Password: [Google Account → Security → 2-Step Verification → App Passwords](#).

■ The Feature

Portfolio data entered in the Builder page must be available on the Template page, Preview page, and everywhere else — without passing it through props at every level. React Context solves this.

■■ Technology: React Context API

React Context is a built-in mechanism to share state globally across a component tree without prop drilling (passing props through every intermediate component). Buildfolio uses two Context providers: AuthContext and PortfolioContext.

The Problem Context Solves

Imagine you have: App → BuilderPage → Step3 → SkillsInput — and SkillsInput needs the user's name. Without Context, you'd pass the name as a prop through every level. With Context, any component anywhere can access it directly.

```
// context/PortfolioContext.jsx
import { createContext, useContext, useState } from "react";
import api from "../utils/api";

const PortfolioContext = createContext(null);

// Provider wraps the app – any child can access portfolio data
export function PortfolioProvider({ children }) {
  const [portfolioData, setPortfolioData] = useState({
    personalInfo: {},
    experiences: [],
    education: [],
    skills: { technical: [], languages: [], tools: [], soft: [] },
    projects: [],
    socialLinks: {},
    achievements: ""
  });

  // Sync to MongoDB Atlas
  const syncToServer = async () => {
    await api.put("/api/portfolio", portfolioData);
  };

  // Load from MongoDB Atlas
  const loadFromServer = async () => {
    const { data } = await api.get("/api/portfolio");
    setPortfolioData(data);
  };

  return (
```


■ The Feature

Buildfolio is live on the internet at `buildfolio-one.vercel.app`. Every time you push code to GitHub, both the frontend and backend automatically redeploy. No manual server management needed.

▲ Technology: Vercel (Frontend Hosting)

Vercel is the company that created Next.js and is the gold standard for hosting React/Vite frontends. It deploys from GitHub automatically, assigns a free HTTPS domain, and serves files from a global CDN (Content Delivery Network).

CDN	Files are copied to servers worldwide. Users get served from the closest one → fast load times.
Auto-deploy	Connect GitHub repo → every push to main branch triggers a new deployment automatically.
Env Variables	Set <code>VITE_API_URL</code> in Vercel dashboard → injected at build time → available as <code>import.meta.env.VITE_</code>
vercel.json	SPA rewrite rule: all paths (<code>/</code>) → <code>index.html</code> . Without this, refreshing <code>/builder</code> returns 404.
Free tier	Unlimited personal projects, custom domains, HTTPS — all free.

```
// vercel.json – required for React SPA routing
{
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

■ Technology: Render (Backend Hosting)

Render hosts the Express backend. Like Vercel, it auto-deploys from GitHub. It provisions a real server (not just static files), so it can run Node.js, connect to databases, and handle API requests.

render.yaml	Declarative config file: specify build command (<code>npm install</code>), start command (<code>npm start</code>), environment.
Free tier caveat	Free Render services spin down after 15 minutes of inactivity → first request after idle takes ~30s to wait
Environment vars	Set <code>MONGO_URI</code> , <code>JWT_SECRET</code> , <code>GEMINI_API_KEY</code> in Render dashboard → available as <code>process.env</code>

■ Technology: MongoDB Atlas (Cloud Database)

MongoDB Atlas is the cloud-hosted version of MongoDB. Instead of running a database on your server (complex, needs maintenance), Atlas manages it for you. The Express backend connects via a `MONGO_URI` connection string.

How to Connect

```
// server.js – Connect to MongoDB Atlas
const mongoose = require("mongoose");

mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log("MongoDB Atlas connected"))
  .catch(err => console.error("DB connection failed:", err));

// MONGO_URI format:
// mongodb+srv://username:password@cluster0.xxxxx.mongodb.net/buildfolio?retryWrites=true
```

■ CORS — Cross-Origin Resource Sharing

Browsers block JavaScript from making requests to a different domain than the page it's on. This is the Same-Origin Policy. CORS is a mechanism that lets the server say 'I allow requests from this specific origin.'

```
// CORS configured to ONLY allow the Vercel frontend
app.use(cors({
  origin: process.env.FRONTEND_URL, // "https://buildfolio-one.vercel.app"
  credentials: true // allow cookies/auth headers
}));
// Any request from a different domain gets a CORS error – attacker blocked.
```

■ Environment Variables

Secrets (passwords, API keys, JWT secret) must never be hardcoded in source code — GitHub is public. dotenv loads them from a .env file locally; on Render and Vercel they're set in the dashboard.

```
# backend/.env – NEVER commit this file to GitHub
MONGO_URI=mongodb+srv://user:pass@cluster0.mongodb.net/buildfolio
JWT_SECRET=a_long_random_secret_string_here
GEMINI_API_KEY=AIzaSy...
FRONTEND_URL=https://buildfolio-one.vercel.app

# In code, access via:
process.env.JWT_SECRET
process.env.GEMINI_API_KEY
```

■ **Note:** Add .env to your .gitignore file. Always. Also never log process.env values — they could appear in error logs accessible to others.

■ Security Checklist for Buildfolio

- Passwords hashed with bcryptjs (cost factor 12)
- JWT tokens expire in 7 days — not forever
- Password reset tokens expire in 1 hour
- CORS restricted to specific frontend domain
- All secrets in environment variables — not in code
- All portfolio/export/AI routes require valid JWT
- Mongoose schema validation prevents malformed data
- express.json() body size limits prevent payload attacks
- Consider adding rate limiting (express-rate-limit) on auth routes
- Consider input sanitization (express-validator) for XSS prevention

You now understand every technology in Buildfolio. Here's a structured path to deepen your knowledge and become a confident full-stack developer:

Phase 1 — Frontend Foundations (2–4 weeks)

- JavaScript ES6+: arrow functions, destructuring, spread, async/await, Promises
- React fundamentals: components, hooks (useState, useEffect, useContext, useRef)
- React Hook Form: register, validate, useFieldArray for dynamic lists
- TailwindCSS: layout (flexbox, grid), responsive prefixes (md:, lg:), hover/focus states
- React Router DOM v7: BrowserRouter, Routes, Route, Link, useNavigate, useParams

Phase 2 — Backend & APIs (2–4 weeks)

- Node.js: event loop, modules (require/import), npm, package.json scripts
- Express.js: routing, middleware, req/res objects, error handling
- REST API design: GET/POST/PUT/DELETE, status codes, JSON responses
- Mongoose: Schema, Model, CRUD (create, findOne, findOneAndUpdate, deleteOne)
- Authentication: JWT (sign, verify), bcryptjs (hash, compare), auth middleware

Phase 3 — Database & Cloud (1–2 weeks)

- MongoDB Atlas: cluster setup, IP whitelist, connection string, indexes
- Mongoose advanced: population (ref), virtuals, pre/post hooks, validators
- Deployment: Vercel (frontend), Render (backend), environment variables
- DNS & HTTPS: custom domains, SSL certificates (automatic on Vercel/Render)

Phase 4 — AI Integration & Advanced (ongoing)

- Prompt engineering: system prompts, temperature, JSON mode, output formatting
- Google Gemini API: REST calls, generationConfig, handling streaming responses
- React Context vs Zustand vs Redux: when to use each state management solution
- Security: CORS, rate limiting, input validation, SQL/NoSQL injection prevention
- Testing: Jest (unit), React Testing Library (component), Supertest (API)

Recommended Free Resources

React	react.dev (official docs — excellent interactive tutorials)
Node/Express	nodejs.org/docs + expressjs.com/en/guide
MongoDB	learn.mongodb.com (free MongoDB University courses)
JavaScript	javascript.info (the best free JS textbook online)
TailwindCSS	tailwindcss.com/docs (interactive playground built in)
JWT	jwt.io (visual decoder + debugger + library list)
Gemini API	aistudio.google.com (free API key + live playground)
Full-Stack	The Odin Project (theodinproject.com) — free end-to-end curriculum

You built Buildfolio. Now you understand every layer of it.

That puts you ahead of most developers who just use frameworks without knowing why.

buildfolio-one.vercel.app	github.com/Mdhummad/buildfolio	Buildfolio Tech Guide
---------------------------	--------------------------------	-----------------------